Package 'ndm'

September 12, 2026

Title Neural Disease Modeling Software

Version 0.5.0

Author Connor T. Jerzak [aut, cre]

Maintainer Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

Description Provides an R-first interface for neural disease modeling with a reticulate-managed JAX backend. The package includes backend bootstrap helpers, built-in epidemiological model specifications, TFRecord-consuming data interfaces, package-managed runtime helpers, and package-native orchestration helpers for model execution.

Depends R (>= 4.1.0)

Imports bestNormalize,
data.table,
digest,
ndmdatasets (>= 0.3.0),
rlang,
reticulate,
utils

Suggests fastmatch,
knitr,
rmarkdown,
rrapply,
testthat (>= 3.0.0),
yaml,
zip,
zoo

VignetteBuilder knitr

License AGPL-3

Encoding UTF-8

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

Contents

ndm-package 2
ndm_bootstrap_multidisease_tfrecords 3
ndm_bootstrap_real_tfrecords 5

ndm_bootstrap_sim_tfrecords 6
ndm_build_model 7
ndm_check_backend 9
ndm_create_config 11
ndm_create_real_run_config 14
ndm_create_tfrecord_parser 19
ndm_extract_temporal_latents 20
ndm_model_spec_from_structure 21
ndm_model_spec_presets 23
ndm_new_runtime_env 25
ndm_predict 26
ndm_prepare_runtime 27
ndm_print 28
ndm_save_model 29
ndm_tfrecord_fields_real 30
ndm_tf_batch_to_r 31

Index **33**

`ndm-package` *Neural Disease Modeling Software*

Description

`ndm` is an R-first package for neural disease modeling with a reticulate-managed JAX backend. The package currently provides backend setup, epidemic model spec import/export, TFRecord readers, package-managed runtime helpers, package-native runtime/model helpers, package-native run APIs, and artifact helpers for saving and restoring trained models.

Details

The current implementation preserves both historical model families, `"DecoderOnly"` and `"NeuralODE"`, while defaulting to `"DecoderOnly"`. The supported backbone is the transformer path only. Latent attention and Mamba are intentionally excluded.

The package treats the structured `ndm_model_spec` object as the canonical internal representation and supports `.tex` import/export for compatibility with the original epidemic specification workflow.

For executable workflows rather than dry-run previews, install the helper packages listed under `Suggests`, initialize the backend with `ndm_initialize_backend`, and align that backend with `NDM_SOFTWARE_CONDA_ENV` or `NDM_CONDA_ENV` when using the package-native runner helpers. When both variables are unset, the runner layer currently falls back to `jax_cpu` on macOS and `ndm_software_env` elsewhere.

Key functions

- `ndm_create_config`
- `ndm_build_backend`
- `ndm_initialize_backend`
- `ndm_model_spec`
- `ndm_model_spec_from_tex`

- ndm_model_spec_to_tex
- ndm_read_tfrecord_dataset
- ndm_load_tfrecord_bundle
- ndm_prepare_runtime
- ndm_prepare_data
- ndm_build_model
- ndm_train
- ndm_predict
- ndm_loss
- ndm_fit
- ndm_create_real_run_config
- ndm_run_real
- ndm_run_sim
- ndm_run_multidisease
- ndm_save_model
- ndm_load_model
- ndm_resume_training

Author(s)

Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

`ndm_bootstrap_multidisease_tfrecords` *Bootstrap canonical multidisease TFRecords by BaseID*

Description

Builds one serial, canonical real-schema TFRecord pair per multidisease 'BaseID'. The bootstrap and training paths share the same raw-data loader, table adapter, dataset specification, source fingerprint, data seed, and complete-location support contract.

Usage

```
ndm_bootstrap_multidisease_tfrecords(
  project_root = getwd(),
  analysis_name = "RealLatest",
  grid = NULL,
  grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  base_ids = NULL,
  tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords", analysis_name),
  disease_names,
  data_format = "IHME",
  outcome_metric = "CountValue",
  data_subset = "all",
```

```
  lookahead = 12L,
  min_anchoring_time = 4L,
  training_target_horizon = NULL,
  producer = NULL,
  overwrite = FALSE,
  dry_run = FALSE,
  covariate_panel_file = NULL,
  covariate_manifest_file = NULL
)
```

Arguments

project_root	Project root containing multidisease data and grids.
analysis_name	Analysis label used for default paths.
grid	Optional in-memory multidisease grid.
grid_file	Optional multidisease grid CSV path.
base_ids	Optional BaseIDs to build, in requested order.
tfrecord_dir	Canonical TFRecord output directory.
disease_names	Non-empty disease names passed to the multidisease loader.
data_format	Multidisease source format: ‘"IHME"‘, ‘"WHO"‘, or ‘"Tycho"‘.
outcome_metric	Outcome column name.
data_subset	Multidisease location subset.
lookahead	Tensor lookahead. This remains distinct from a grid row’s shorter ‘evaluationHorizon‘.
min_anchoring_time	Minimum training anchor time.
training_target_horizon	Optional positive number of consecutive post-anchor targets that must be finite for an anchor to be eligible for training. It must not exceed ‘lookahead‘; a same-named grid column is used when this argument is ‘NULL‘.
producer	Non-empty named producer metadata. Required to publish. For environment-bound training, use ‘list(contract = "<contract>")‘.
overwrite	Whether to replace an existing complete pair.
dry_run	Whether to return the plan without loading data or writing.
covariate_panel_file	Optional WHO annual covariate panel CSV with unique ‘location_id‘/‘year‘ keys and numeric feature columns. It must be supplied together with ‘covariate_manifest_file‘.
covariate_manifest_file	Optional covariate manifest CSV. Its unique, ordered ‘feature_name‘ column must exactly name the panel’s non-key columns; additional columns are retained as provenance metadata.

Value

A data frame with one row per BaseID. ‘status‘ is ‘"planned"‘ for dry runs, ‘"written"‘ after publication, or ‘"skipped_existing"‘ only after an existing pair passes full contract and readability validation.

ndm_bootstrap_real_tfrecords *Bootstrap canonical real-data TFRecords by BaseID*

Description

Materializes one canonical real-data train/inference artifact pair per 'BaseID', with the same cross-process lock and canonical-row selection used by [ndm_bootstrap_sim_tfrecords()].

Usage

```
ndm_bootstrap_real_tfrecords(
  project_root = getwd(),
  analysis_name = "RealApril15",
  grid = NULL,
 grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  base_ids = NULL,
 tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords", analysis_name),
  raw_data_dir = file.path("Data", "MainData"),
  outcome_metric = "inc_death",
  data_subset = "high_income",
  producer = NULL,
  parallel_workers = 1L,
  overwrite = FALSE,
  dry_run = FALSE
)
```

Arguments

project_root	Working project directory used to resolve relative grid and TFRecord paths.
analysis_name	Analysis label used to derive default real-data paths.
grid	Optional in-memory real-data grid.
grid_file	Optional CSV path for the real-data grid.
base_ids	Optional integer vector of canonical 'BaseID' values to materialize. When 'NULL', bootstrap all 'BaseID's in raw grid order of first appearance.
tfrecord_dir	Output directory for canonical real-data TFRecords.
raw_data_dir	Directory containing the real-data source tables.
outcome_metric	Outcome metric passed to 'ndmdatasets'.
data_subset	Real-data subset passed to 'ndmdatasets'.
producer	Nonempty named producer metadata stored in every artifact manifest. Required for publication; dry runs may omit it.
parallel_workers	Positive integer number of forked BaseID workers on Linux. Inputs are prepared before forking. macOS and Windows use the deterministic serial path because TensorFlow/JAX runtimes are not fork-safe there.
overwrite	Logical scalar controlling whether existing canonical TFRecords should be regenerated.
dry_run	Logical scalar indicating whether to return the canonical write plan without writing TFRecords.

Value

A canonical per-‘BaseID‘ write-plan data frame with a ‘status‘ column.

`ndm_bootstrap_sim_tfrecords` *Bootstrap canonical simulation TFRecords by BaseID*

Description

This helper materializes canonical simulation TFRecords without going through the legacy ‘outer‘ scheduling path. It reads the simulation grid in raw CSV or data-frame order, builds one canonical write plan row per ‘BaseID‘, and optionally writes those TFRecords with a per-‘BaseID‘ cross-process lock.

Usage

```
ndm_bootstrap_sim_tfrecords(
  project_root = getwd(),
  analysis_name = "BigSimsLatest",
  grid = NULL,
  grid_file = file.path("Data", "RunGrids", "SimGrids", sprintf("SimGrid_%s.csv",
    analysis_name)),
  base_ids = NULL,
  tfrecord_dir = file.path("Data", "RunTFRecords", "SimTFRecords", analysis_name),
  producer = NULL,
  parallel_workers = 1L,
  overwrite = FALSE,
  dry_run = FALSE
)
```

Arguments

`project_root`	Working project directory used to resolve relative grid and TFRecord paths.
`analysis_name`	Analysis label used to derive default grid and TFRecord paths.
`grid`	Optional in-memory simulation grid. When supplied, ‘grid_file‘ must be ‘NULL‘.
`grid_file`	Optional CSV path for the simulation grid.
`base_ids`	Optional integer vector of canonical ‘BaseID‘ values to materialize. When ‘NULL‘, bootstrap all ‘BaseID‘s in raw grid order of first appearance.
`tfrecord_dir`	Output directory for canonical TFRecords.
`producer`	Nonempty named producer metadata stored in every artifact manifest. Required for publication; dry runs may omit it.
`parallel_workers`	Positive integer number of forked BaseID workers on Linux. Inputs are prepared before forking. macOS and Windows use the deterministic serial path because TensorFlow/JAX runtimes are not fork-safe there.
`overwrite`	Logical scalar controlling whether existing canonical TFRecords should be regenerated.
`dry_run`	Logical scalar indicating whether to return the canonical write plan without writing TFRecords.

Value

A data frame with one row per canonical ‘BaseID‘ and columns ‘BaseID‘, ‘selected_rows‘, ‘canonical_row‘, ‘artifact_n_samples_train‘, ‘train_file‘, ‘inference_file‘, and ‘status‘.

`ndm_build_model` *Build and train models with package-managed runtime stages*

Description

These wrappers layer a small R API over package-managed Phase 1 model build and training stages.

Usage

```
ndm_build_model(
  runtime_env,
  model_type = c("DecoderOnly", "NeuralODE"),
  model_spec = NULL,
  backbone = "transformer",
  runtime_globals = list()
)

## S3 method for class 'ndm_model'
print(x, ...)

ndm_train(x, run_define = TRUE, run_loop = TRUE)

## S3 method for class 'ndm_trained_model'
print(x, ...)

ndm_fit(
  config = ndm_create_config(),
  model_spec = NULL,
  data_generator = c("sim", "real", "multidisease"),
  runtime_globals = list(),
  data_globals = list(),
  build_globals = list(),
  train_globals = list(),
  run_define = TRUE,
  run_loop = TRUE
)
```

Arguments

`runtime_env`	Runtime environment containing loaded runtime helpers and data globals.
`model_type`	Model family to build. Either ‘"DecoderOnly"‘ or ‘"NeuralODE"‘.
`model_spec`	Optional ‘ndm_model_spec‘ object used to override the model TeX specification supplied to the runtime model builder.
`backbone`	Backbone family. Phase 1 supports ‘"transformer"‘ only.

`runtime_globals`
: Named list of additional globals assigned during the relevant runtime stage. ‘ndm_build_model()‘ uses them before building the model, while ‘ndm_fit()‘ uses them before sourcing the runtime.

`x`
: Either an ‘ndm_model‘, an ‘ndm_trained_model‘, or a prepared runtime environment, depending on the function being called.

`...`
: Unused; included for S3 method compatibility.

`run_define`
: Logical scalar indicating whether the training definition script should be sourced.

`run_loop`
: Logical scalar indicating whether the training loop script should be sourced.

`config`
: An object of class ‘ndm_config‘, usually created by ‘ndm_create_config()‘.

`data_generator`
: Which data workflow to prepare before calling ‘ndm_build_model()‘ inside ‘ndm_fit()‘.

`data_globals`
: Named list of globals assigned before data preparation in ‘ndm_fit()‘.

`build_globals`
: Named list of globals assigned before building the model in ‘ndm_fit()‘.

`train_globals`
: Named list of globals assigned immediately before training in ‘ndm_fit()‘.

Details

These functions assume the backend has already been initialized with ‘ndm_initialize_backend()‘ and that ‘runtime_env‘ already contains the upstream runtime and data globals required by the selected workflow. ‘ndm_train()‘ requires ‘rrapply‘; it also requires ‘zip‘ when checkpointing is enabled and ‘zoo‘ for multidisease training. Simulation preparation only requires ‘zoo‘ when ‘SkipTfRecords = TRUE‘ selects the in-memory compatibility path documented on [ndm_prepare_runtime()].

Value

‘ndm_build_model()‘ returns an object of class ‘ndm_model‘. ‘ndm_train()‘ and ‘ndm_fit()‘ return an object of class ‘ndm_trained_model‘.

‘print.ndm_model()‘ prints a brief summary of an ‘ndm_model‘ object and invisibly returns ‘x‘.

‘print.ndm_trained_model()‘ prints a brief summary of an ‘ndm_trained_model‘ object and invisibly returns ‘x‘.

Examples

```
cfg <- ndm_create_config()
spec <- ndm_model_spec()
## Not run:
runtime_env <- ndm_prepare_runtime(cfg)
ndm_prepare_data(runtime_env, generator = "sim")
model <- ndm_build_model(runtime_env, model_spec = spec)
trained <- ndm_train(model)

## End(Not run)
```

ndm_check_backend *Provision and initialize the Python backend*

Description

These helpers manage the reticulate-backed Python environment used by ndm. Use ‘ndm_build_backend()‘ to provision dependencies, ‘ndm_check_backend()‘ to verify availability, and ‘ndm_initialize_backend()‘ to import the active JAX stack into R.

Usage

```
ndm_check_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  modules = c("jax", "numpy", "optax", "equinox", "diffrax")
)

ndm_build_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  python_version = "3.13",
  include_tensorflow = TRUE,
  extra_packages = c("optax", "equinox", "diffrax")
)

ndm_initialize_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  conda_env_required = TRUE,
  float_type = c("32", "64"),
  import_tensorflow = TRUE,
  compute_backend = c("auto", "cpu", "gpu"),
  force_to_gpu = NULL,
  gpu_mem_frac = NULL
)

ndm_backend_modules()
```

Arguments

conda_env	Name of the conda environment that should contain the JAX runtime.
conda	Conda executable to use. ‘"auto"‘ delegates discovery to reticulate.
modules	Character vector of Python module names that must be importable for the backend to be considered healthy.
python_version	Python version requested when creating the conda environment.
include_tensorflow	Logical scalar indicating whether TensorFlow should be installed alongside JAX.
extra_packages	Additional Python packages to install with ‘pip‘.
conda_env_required	Passed through to ‘reticulate::use_condaenv()‘ as the ‘required‘ flag.

`float_type` Floating point precision used when configuring JAX. Either ‘"32"‘ or ‘"64"‘.

`import_tensorflow`
Logical scalar indicating whether TensorFlow should be imported when it is available in the selected environment.

`compute_backend`
Compute device policy. ‘"auto"‘ selects a supported JAX GPU when an isolated pre-import probe succeeds and otherwise uses CPU; ‘"cpu"‘ and ‘"gpu"‘ require that backend explicitly. macOS supports CPU execution only under this contract.

`force_to_gpu` Deprecated logical alias for ‘compute_backend‘. ‘TRUE‘ maps to ‘"gpu"‘ and ‘FALSE‘ maps to ‘"cpu"‘. Supplying conflicting old and new controls is an error.

`gpu_mem_frac` Optional finite GPU memory fraction in ‘(0, 1]‘. It is rejected whenever backend selection resolves to CPU.

Details

When you want backend helpers and the package-native ‘ndm_run_*()‘ runners to target the same conda environment, set ‘NDM_SOFTWARE_CONDA_ENV‘ (or ‘NDM_CONDA_ENV‘) once and pass that value into these helpers. The runner layer currently falls back to ‘jax_cpu‘ on macOS and ‘ndm_software_env‘ elsewhere when neither environment variable is set.

Value

‘ndm_check_backend()‘ returns ‘TRUE‘ invisibly when the requested environment and modules are available. It returns ‘NULL‘ after printing a diagnostic message when the backend is not ready.

‘ndm_build_backend()‘ invisibly returns the conda environment name after attempting to provision the requested packages.

‘ndm_initialize_backend()‘ returns an object of class ‘ndm_backend‘. The returned list contains imported Python modules, the configured float type, requested and resolved compute backends, selected-device provenance, and compatibility shims used by the legacy runtime.

‘ndm_backend_modules()‘ returns the currently initialized ‘ndm_backend‘ object.

Examples

```
ndm_check_backend()

## Not run:
ndm_build_backend()

## End(Not run)

## Not run:
backend <- ndm_initialize_backend()
backend$default_backend

## End(Not run)

## Not run:
ndm_initialize_backend()
ndm_backend_modules()

## End(Not run)
```

ndm_create_config *Create runtime configuration objects*

Description

Configuration objects collect the runtime options that are threaded through the higher-level orchestration helpers such as ‘ndm_prepare_runtime()‘ and ‘ndm_fit()‘.

Usage

```
ndm_create_config(
  model_type = c("DecoderOnly", "NeuralODE"),
  backbone = "transformer",
  float_type = c("32", "64"),
  force_to_gpu = NULL,
  resave_tfrecords = FALSE,
  gpu_mem_frac = NULL,
  neuralode_local_latent_dim = NULL,
  neuralode_global_latent_dim = NULL,
  neuralode_init_state_logit_offset = NULL,
  neuralode_init_state_logit_scale_max = Inf,
  neuralode_optim_solver = c("tsit5", "dopri8"),
  neuralode_optim_dt0 = 0.1,
  neuralode_optim_controller = c("pid", "constant"),
  neuralode_optim_rtol = 1e-05,
  neuralode_optim_atol = 1e-07,
  neuralode_train_integrator = c("auto", "fixed_rk4", "diffrax"),
  neuralode_train_substeps = NULL,
  enable_kv_cache = TRUE,
  enable_kv_cache_training = TRUE,
  inference_mc_draws = 5L,
  observation_scale_floor = 1e-05,
  initial_observation_scale = 1,
  neuralode_variational = TRUE,
  neuralode_kl_weight = 1,
  neuralode_mean_loss_weight = 0,
  training_objective = c("student_t_nll", "scaled_mse"),
  outcome_loss_scale = NULL,
  compute_backend = c("auto", "cpu", "gpu"),
  transformer_compute_dtype = c("auto", "native", "bfloat16", "float32"),
  transformer_activation_checkpointing = FALSE,
  donate_training_state = TRUE,
  ...
)

## S3 method for class 'ndm_config'
print(x, ...)
```

Arguments

model_type	Model family metadata. Use ‘"DecoderOnly"‘ or ‘"NeuralODE"‘.

`backbone` Backbone family. Phase 1 supports ‘"transformer"‘ only.

`float_type` Floating point precision used when initializing the backend. Use ‘"32"‘ or ‘"64"‘.

`force_to_gpu` Deprecated logical alias for ‘compute_backend‘. ‘TRUE‘ maps to ‘"gpu"‘ and ‘FALSE‘ maps to ‘"cpu"‘.

`resave_tfrecords`
Logical scalar preserved for compatibility. Public training APIs require ‘FALSE‘; prepare canonical inputs before training.

`gpu_mem_frac` Optional finite GPU memory fraction in ‘(0, 1]‘. It is invalid when ‘compute_backend = "cpu"‘ and rejected at initialization if ‘"auto"‘ resolves to CPU.

`neuralode_local_latent_dim`
Optional local NeuralODE latent width. When ‘NULL‘, runtime code resolves this to ‘ModelDims‘.

`neuralode_global_latent_dim`
Optional global NeuralODE latent width. When ‘NULL‘, runtime code resolves this to ‘ModelDims‘.

`neuralode_init_state_logit_offset`
Optional init-state logit offset vector for the NeuralODE state simplex initializer.

`neuralode_init_state_logit_scale_max`
Scalar cap applied to the NeuralODE init-state logit scale.

`neuralode_optim_solver`
NeuralODE optimization solver name. Use ‘"tsit5"‘ or ‘"dopri8"‘.

`neuralode_optim_dt0`
Initial step size used by the NeuralODE optimization solve.

`neuralode_optim_controller`
NeuralODE optimization stepsize controller. Use ‘"pid"‘ or ‘"constant"‘.

`neuralode_optim_rtol`
Relative tolerance used when ‘neuralode_optim_controller = "pid"‘.

`neuralode_optim_atol`
Absolute tolerance used when ‘neuralode_optim_controller = "pid"‘.

`neuralode_train_integrator`
Integrator for the NeuralODE solves inside the training loss and gradient. ‘"fixed_rk4"‘ runs a fixed-step RK4 scan on the integer save grid, differentiated with plain reverse-mode AD, which costs a fraction of the adaptive solve’s adjoint. ‘"diffrax"‘ uses the adaptive ‘neuralode_optim_*‘ solve for training as well. ‘"auto"‘ (the default) uses ‘"fixed_rk4"‘ for structures without global-neural terms and ‘"diffrax"‘ for structures with them, whose dynamics are stiff at initialisation. Prediction and analytics always use the adaptive solve.

`neuralode_train_substeps`
Number of RK4 substeps per unit time for ‘neuralode_train_integrator = "fixed_rk4"‘, or ‘NULL‘ (the default) to choose automatically: two for structures with global-neural terms, whose compartments move quickly, and one otherwise. Raise it if training dynamics become stiff.

`enable_kv_cache`
Logical scalar controlling DecoderOnly KV caching. The default enables the faster cached inference path.

`enable_kv_cache_training`
Logical scalar controlling KV caching during DecoderOnly gradient training. The default is ‘TRUE‘: the cached training rollout is gradient-equivalent to the

un-cached reference (verified to float32 tolerance across production topologies, left padding, and mask holes) and several times faster per training step at production batch sizes. Set 'FALSE' to reproduce historical un-cached training exactly.

`inference_mc_draws`
: Positive integer number of posterior draws requested for inference. Deterministic NeuralODE and DecoderOnly inference reduce this to one effective draw downstream.

`observation_scale_floor`
: Finite positive lower bound added to learned observation scales.

`initial_observation_scale`
: Finite positive initial observation-scale value on the outcome scale. The default is '1', matching the unit-scale initialization used before this control became configurable. It must be greater than 'observation_scale_floor'.

`neuralode_variational`
: Logical scalar. When 'FALSE', NeuralODE training uses deterministic posterior means instead of stochastic variational samples.

`neuralode_kl_weight`
: Non-negative scalar KL multiplier used only when 'neuralode_variational = TRUE'.

`neuralode_mean_loss_weight`
: Non-negative multiplier for an auxiliary masked mean-squared-error term applied to NeuralODE forecasts alongside the Student-t likelihood.

`training_objective`
: Training loss for observed outcomes. The default, '"student_t_nll"', preserves the probabilistic likelihood. Use '"scaled_mse"' for deterministic scale-normalized squared error.

`outcome_loss_scale`
: Optional finite positive scalar or outcome-length vector used to normalize residuals for 'training_objective = "scaled_mse"'. It is required for scaled MSE and otherwise must be 'NULL'.

`compute_backend`
: Compute device policy: '"auto"' selects a supported JAX GPU when available and otherwise CPU, while '"cpu"' and '"gpu"' require the named backend.

`transformer_compute_dtype`
: Transformer activation precision. '"auto"' uses BF16 for FP32 CUDA models and native precision otherwise. '"native"', '"bfloat16"', and '"float32"' are explicit overrides. Master parameters, optimizer moments, prediction heads, and ODE arithmetic retain 'float_type'.

`transformer_activation_checkpointing`
: Recompute transformer sublayer intermediates during backpropagation to reduce activation memory. Defaults to 'FALSE': at typical model sizes the saved activations are small, while the extra forward pass costs measurable training time. Enable it when activation memory is the binding constraint.

`donate_training_state`
: Allow compiled updates to reuse parameter and optimizer buffers. Rejected updates return the original values in fresh usable state handles for diagnostics.

`...`
: Unused; included for S3 method compatibility.

`x`
: An 'ndm_config' object.

Value

'ndm_create_config()' returns an object of class 'ndm_config'.

'print.ndm_config()' prints a brief summary of an 'ndm_config' object and invisibly returns 'x'.

Examples

```
cfg <- ndm_create_config()
print(cfg)
```

---

ndm_create_real_run_config *Package-native run configurations and workflow runners*

Description

Package-native configuration constructors and workflow runners for the self-contained real, simulation, and multidisease execution paths.

Usage

```
ndm_create_real_run_config(
  project_root = getwd(),
  analysis_name = "RealApril15",
  grid = NULL,
  grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  outer = 3L,
  run_seed = NULL,
  force_to_gpu = NULL,
  gpu_mem_frac = NULL,
  enable_kv_cache = TRUE,
  enable_kv_cache_training = TRUE,
  inference_mc_draws = 5L,
  observation_scale_floor = 1e-05,
  initial_observation_scale = 1,
  neuralode_variational = TRUE,
  neuralode_kl_weight = 1,
  neuralode_mean_loss_weight = 0,
  training_objective = c("student_t_nll", "scaled_mse"),
  outcome_loss_scale = NULL,
  n_checkpoints = 1L,
  max_sgd_steps = NULL,
  prior_sd_multiplier = 1,
  solver_profile = "default",
  model_type = NULL,
  respect_grid_model_type = TRUE,
  resave_tfrecords = FALSE,
  tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords",
    analysis_name),
```

```
  raw_data_dir = file.path("Data", "MainData"),
  outcome_metric = "inc_death",
  data_subset = "high_income",
  dry_run = FALSE,
  compute_backend = c("auto", "cpu", "gpu")
)

ndm_create_sim_run_config(
  project_root = getwd(),
  analysis_name = "BigSimsLatest",
  grid = NULL,
 grid_file = file.path("Data", "RunGrids", "SimGrids", sprintf("SimGrid_%s.csv",
    analysis_name)),
  outer = 1L,
  run_seed = NULL,
  force_to_gpu = NULL,
  gpu_mem_frac = NULL,
  enable_kv_cache = TRUE,
  enable_kv_cache_training = TRUE,
  inference_mc_draws = 5L,
  observation_scale_floor = 1e-05,
  initial_observation_scale = 1,
  neuralode_variational = TRUE,
  neuralode_kl_weight = 1,
  neuralode_mean_loss_weight = 0,
  training_objective = c("student_t_nll", "scaled_mse"),
  outcome_loss_scale = NULL,
  n_checkpoints = 1L,
  max_sgd_steps = NULL,
  prior_sd_multiplier = 1,
  solver_profile = "default",
  model_type = NULL,
  respect_grid_model_type = TRUE,
  resave_tfrecords = FALSE,
  tfrecord_dir = file.path("Data", "RunTFRecords", "SimTFRecords",
    analysis_name),
  dry_run = FALSE,
  compute_backend = c("auto", "cpu", "gpu")
)

ndm_create_multidisease_run_config(
  project_root = getwd(),
  analysis_name = "RealLatest",
  grid = NULL,
 grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  outer = 1L,
  run_seed = NULL,
  force_to_gpu = NULL,
  gpu_mem_frac = NULL,
  enable_kv_cache = TRUE,
  enable_kv_cache_training = TRUE,
```

```
  inference_mc_draws = 5L,
  observation_scale_floor = 1e-05,
  initial_observation_scale = 1,
  neuralode_variational = TRUE,
  neuralode_kl_weight = 1,
  neuralode_mean_loss_weight = 0,
  training_objective = c("student_t_nll", "scaled_mse"),
  outcome_loss_scale = NULL,
  n_checkpoints = 1L,
  max_sgd_steps = NULL,
  prior_sd_multiplier = 1,
  solver_profile = "default",
  model_type = NULL,
  respect_grid_model_type = TRUE,
  resave_tfrecords = FALSE,
  tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords",
    analysis_name),
  outcome_metric = "CountValue",
  data_subset = "all",
  disease_names = c("Covid", "Flu"),
  data_format = "IHME",
  dry_run = FALSE,
  covariate_panel_file = NULL,
  covariate_manifest_file = NULL,
  compute_backend = c("auto", "cpu", "gpu")
)

ndm_run_real(config = ndm_create_real_run_config())

ndm_run_sim(config = ndm_create_sim_run_config())

ndm_run_multidisease(config = ndm_create_multidisease_run_config())
```

Arguments

project_root	Working project directory used for grids, TFRecords, and outputs.
analysis_name	Analysis label threaded through output paths.
grid	Optional in-memory grid data frame. When supplied, grid_file must be NULL.
grid_file	Optional CSV path for the run grid.
outer	Integer vector of outer-iteration rows to execute.
run_seed	Optional non-negative integer used for model initialization, training Monte Carlo keys, and inference randomness. When NULL, the legacy outer-row-derived seed is retained.
force_to_gpu	Deprecated logical alias for compute_backend. TRUE maps to "gpu" and FALSE maps to "cpu".
compute_backend	Compute device policy. "auto" selects a supported JAX GPU when available and otherwise CPU; "cpu" and "gpu" require the named backend.
gpu_mem_frac	Optional finite GPU memory fraction in (0, 1]. It is invalid with compute_backend = "cpu" and rejected during initialization if "auto" resolves to CPU.

enable_kv_cache
: Logical scalar controlling DecoderOnly KV caching. The default enables caching for inference.

enable_kv_cache_training
: Logical scalar controlling caching during DecoderOnly gradient training. It defaults to TRUE: cached training is gradient-equivalent to the un-cached rollout and several times faster per step. Set FALSE to reproduce historical un-cached training exactly.

inference_mc_draws
: Positive integer number of requested posterior inference draws. Deterministic NeuralODE and DecoderOnly paths use one effective draw downstream.

observation_scale_floor
: Finite positive lower bound added to learned observation scales.

initial_observation_scale
: Finite positive initial observation-scale value on the outcome scale. The default is 1, matching the unit-scale initialization used before this control became configurable. It must be greater than observation_scale_floor.

neuralode_variational
: Logical scalar controlling variational NeuralODE training and inference. It has no effect on DecoderOnly models.

neuralode_kl_weight
: Finite non-negative NeuralODE KL multiplier. Zero disables KL evaluation while preserving variational sampling.

neuralode_mean_loss_weight
: Finite non-negative multiplier for the NeuralODE auxiliary masked mean-squared-error term.

training_objective
: Training loss for observed outcomes. The default "student_t_nll" preserves the probabilistic likelihood; "scaled_mse" selects deterministic scale-normalized squared error.

outcome_loss_scale
: Optional finite positive scalar or outcome-length vector used by training_objective = "scaled_mse". It is required for scaled MSE and otherwise must be NULL.

n_checkpoints
: Positive integer number of checkpoint and analytics states to retain, including the terminal post-update state.

max_sgd_steps
: Optional positive integer training-step cap for explicitly labeled pilot runs.

prior_sd_multiplier
: Finite positive multiplier applied to ODE prior standard deviations.

solver_profile
: ODE optimization profile: "default", "loose", "tight", or "alternative".

model_type
: Optional model family override.

respect_grid_model_type
: Logical scalar indicating whether grid rows may override the requested model type.

resave_tfrecords
: Logical scalar retained for compatibility. Training workflows require FALSE. For real and simulation runs, build canonical artifacts separately with ndm_bootstrap_real_tfrecords(or ndm_bootstrap_sim_tfrecords(). For multidisease runs, use ndm_bootstrap_multidisease_

tfrecord_dir
: Optional TFRecord output directory.

raw_data_dir
: Real-data input directory.

`outcome_metric` Outcome metric name for real-data or multidisease runs.

`data_subset` Optional real-data or multidisease subset name.

`disease_names` Character vector of multidisease names.

`data_format` Multidisease data format.

`covariate_panel_file`
Optional WHO annual covariate-panel CSV, keyed by `location_id` and `year`. Supply it together with `covariate_manifest_file`.

`covariate_manifest_file`
Optional CSV whose ordered `feature_name` column defines the covariate-panel feature contract and whose remaining columns carry provenance metadata.

`dry_run` Logical scalar indicating whether the run should stop after resolving paths and grid rows.

`config` A package-native run configuration created by the matching `ndm_create_*_run_config()` helper.

Details

`dry_run = TRUE` resolves paths and selected rows without executing the underlying workflow. Dry runs can use lightweight in-memory grids such as the `BaseID` / `ModelType` preview shown in `README.md`.

Non-dry execution expects Analysis2-compatible grids. The tested simulation grids in this package are produced by `ndmdatasets::ndm_sim_build_grid()` and then augmented with fields such as `paddingMethod`, `lookahead`, `n_time_steps`, `n_inference_batches`, `scaling_batches`, and either `model_spec_name` or `model_tex_loc`. The tested real and multidisease grids include `BaseID`, `ContextLength`, `evaluationTime`, `initialTransform`, `initialNormType`, `paddingMethod`, `OSSType`, `dataInputs`, `ModelType`, `ModelDepth`, `ModelDims`, `nSamplesTrain`, `nObsInference`, `floatType`, and either `model_spec_name` or `model_tex_loc`.

Non-dry runner workflows also require the full execution stack: the `ndmdatasets` package, the helper packages used by the runtime, and a backend environment aligned with `NDM_SOFTWARE_CONDA_ENV`, `NDM_CONDA_ENV`, or the platform-specific fallback used by the runner layer.

Value

The `ndm_create_*_run_config()` helpers return classed lists describing the requested workflow. Each `ndm_run_*()` function returns a dry-run preview when `config$dry_run` is `TRUE` and otherwise the underlying workflow result.

Examples

```
## Not run:
preview_grid <- data.frame(
  BaseID = c(1L, 2L),
  ModelType = c("DecoderOnly", "NeuralODE"),
  stringsAsFactors = FALSE
)

cfg <- ndm_create_sim_run_config(
  project_root = tempdir(),
  grid = preview_grid,
  outer = 1:2,
  dry_run = TRUE
)
```

```
ndm_run_sim(cfg)

## End(Not run)
```

`ndm_create_tfrecord_parser` *Create and read TensorFlow TFRecord datasets*

Description

These wrappers construct parsers for the ndm TFRecord schema and use them to build TensorFlow datasets or eagerly collect batches into R.

Usage

```
ndm_create_tfrecord_parser(field_names, dtype_map = NULL, tensorflow = NULL)

ndm_read_tfrecord_dataset(
  file,
  batch_size,
  field_names,
  dtype_map = NULL,
  max_examples = NULL,
  shuffle = FALSE,
  buffer_scaler = 10L,
  tensorflow = NULL
)

ndm_collect_tfrecord_batches(
  file,
  batch_size,
  field_names,
  dtype_map = NULL,
  max_batches = NULL,
  max_examples = NULL,
  shuffle = FALSE,
  buffer_scaler = 10L,
  tensorflow = NULL
)
```

Arguments

`field_names`	Character vector of TFRecord field names that should be parsed from each example.
`dtype_map`	Optional named vector or list mapping ‘field_names‘ to TensorFlow dtype names or TensorFlow dtype objects.
`tensorflow`	Optional TensorFlow module. When omitted, ndm uses the active backend’s TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time.
`file`	Path to the ‘.tfrecord‘ file to read.

batch_size	Batch size used when constructing the TensorFlow dataset.
max_examples	Optional maximum number of examples to consume from the file before batching.
shuffle	Logical scalar indicating whether the dataset should be shuffled.
buffer_scaler	Integer multiplier used to derive the TensorFlow shuffle buffer size from ‘batch_size‘.
max_batches	Optional maximum number of batches to collect when using ‘ndm_collect_tfrecord_batches()‘.

Details

TFRecord parsing requires TensorFlow to be importable from the active reticulate environment. In practice this usually means provisioning the backend with ‘ndm_build_backend(include_tensorflow = TRUE)‘ and then calling ‘ndm_initialize_backend(import_tensorflow = TRUE)‘ for the same conda environment before reading datasets.

Value

‘ndm_create_tfrecord_parser()‘ returns a parser function compatible with ‘tf$data$Dataset$map()‘. ‘ndm_read_tfrecord_dataset()‘ returns a TensorFlow dataset object. ‘ndm_collect_tfrecord_batches()‘ returns a list of parsed batches.

Examples

```
fields <- ndm_tfrecord_fields_real()
parser <- ndm_create_tfrecord_parser(fields, ndm_tfrecord_dtype_map("real"))
is.function(parser)
```

---

ndm_extract_temporal_latents

Extract temporal latent states for interpretability analyses

Description

‘ndm_extract_temporal_latents()‘ runs the existing prediction path and returns aligned temporal representations for decoder-only and NeuralODE models. ‘ndm_latent_matrix()‘ flattens one extracted representation to a two-dimensional matrix suitable for external analyses such as CCA.

Usage

```
ndm_extract_temporal_latents(
  x,
  batch = NULL,
  seed = 1L,
  inference = TRUE,
  update_state = FALSE,
  as_r = TRUE
)

ndm_latent_matrix(latents, source = "decoder_head_input", features = NULL)
```

Arguments

x	Either an ‘ndm_model‘, an ‘ndm_trained_model‘, or a prepared runtime environment that already contains the required runtime objects.
batch	Optional prediction batch passed through to [ndm_predict()].
seed	Integer seed used to derive the per-example JAX RNG keys.
inference	Logical scalar indicating whether the inference prediction path or training prediction path should be used.
update_state	Logical scalar indicating whether the runtime ‘state‘ object should be updated with the returned state.
as_r	Logical scalar. When ‘TRUE‘, JAX arrays are copied to R arrays.
latents	An object returned by ‘ndm_extract_temporal_latents()‘.
source	Representation to flatten. Supported values include ‘"decoder_head_input"‘, ‘"ode_local_hidden"‘, ‘"ode_global_hidden"‘, ‘"ode_local_features"‘, ‘"ode_global_features"‘, and ‘"ode_states"‘.
features	Optional feature names or integer feature positions.

Value

‘ndm_extract_temporal_latents()‘ returns an ‘ndm_temporal_latents‘ list. ‘ndm_latent_matrix()‘ returns a numeric matrix with rows ordered by batch within time.

Examples

```
## Not run:
latents <- ndm_extract_temporal_latents(trained, batch = batch_l_cal)
decoder_x <- ndm_latent_matrix(latents, "decoder_head_input")
ode_x <- ndm_latent_matrix(latents, "ode_local_hidden")

## End(Not run)
```

---

ndm_model_spec_from_structure

Inspect and create epidemic model specifications from structured declarations

---

Description

‘ndm_model_spec_from_structure()‘ builds an ‘ndm_model_spec‘ from a structured declaration. The declaration may either describe the full model directly or identify a supported built-in family plus optional ‘family_args‘ that expand parameterized structures into explicit executable equations.

Usage

```
ndm_model_spec_from_structure(
  structure,
  model_type = c("DecoderOnly", "NeuralODE"),
  family_args = NULL
)
```

Arguments

structure	A structured model declaration.
model_type	Model family metadata attached to the returned specification. Either '"DecoderOnly"' or '"NeuralODE"'.
family_args	Optional named list used to expand parameterized family declarations. For structured TB families, 'observation_target' chooses '"incidence"' (default flow into 'i_1') or '"cumulative"' ('i_1'). TB families use 'dynamics_variant = "balanced_incidence"' by default, preserving the original active-progression flow while adding normalized at-risk renewal ('N = 1'), active-disease exit back to susceptibility, latent mortality, and incidence-flow observations. Set 'dynamics_variant = "progression"' for legacy one-way depletion dynamics. Set 'dynamics_variant = "seir"' for the SEIR-style form of 'reports/TBModels/TBDiseaseTypes_v2.md': the constant force of infection 'lambda' is replaced by mass-action incidence 'beta * i_1 / N', an explicit removed compartment 'r_1' is added with active-disease removal 'gamma', and demographic renewal 'mu' sustains endemic incidence. Both 'balanced_incidence' and 'seir' require incidence observations.

Details

For a maintainer-oriented guide to the required fields, parameter-spec shape, TeX contract, and extension workflow, see `vignette("ode-structures", package = "ndm")`.

Value

An object of class 'ndm_model_spec'.

Examples

```
family_spec <- ndm_model_spec_from_structure(
  list(
    family = "tb_b",
    family_args = list(n = 4L)
  ),
  model_type = "NeuralODE"
)
family_spec$state_terms

toy_spec <- ndm_model_spec_from_structure(
  list(
    preset = "toy_slqi",
    description = "Toy four-state latent structure with a holding compartment.",
    states = c("s_l", "l_l", "q_l", "i_l"),
    parameters = list(
      lambda = list(
        transformation = "InvSoftPlus",
        prior_mean = 0.08,
        prior_sd = 0.25
      ),
      c = list(
        transformation = "InvSoftPlus",
        prior_mean = 0.05,
        prior_sd = 0.25
      ),
      h = list(
```

```
        transformation = "InvSoftPlus",
        prior_mean = 0.03,
        prior_sd = 0.25
      )
    ),
    equations = c(
      s_l = "- lambda * s_l",
      l_l = "lambda * s_l - (c + h) * l_l",
      q_l = "h * l_l",
      i_l = "c * l_l"
    ),
    observations = "i_l"
  ),
  model_type = "NeuralODE"
)
toy_spec$parameter_terms
```

---

ndm_model_spec_presets *Inspect and create epidemic model specifications*

Description

The package ships a small registry of built-in compartmental model specifications and also supports importing custom '.tex' model definitions from disk. The structured 'ndm_model_spec' object is the canonical representation used by the runtime wrappers.

Usage

```
ndm_model_spec_presets()

ndm_model_spec(
  preset = NULL,
  compartments = c("seir", "seirs"),
  dynamic_beta = NULL,
  dynamic_global = FALSE,
  multi_outcome = FALSE,
  model_type = c("DecoderOnly", "NeuralODE"),
  time_varying_terms = NULL,
  endogenous_terms = NULL,
  tex_path = NULL,
  family_args = NULL
)

ndm_model_spec_from_tex(path, model_type = c("DecoderOnly", "NeuralODE"))

ndm_model_spec_to_tex(spec, path = NULL)

## S3 method for class 'ndm_model_spec'
print(x, ...)
```

Arguments

preset	Optional preset name from ‘ndm_model_spec_presets()‘. When omitted, the remaining arguments are used to infer a built-in preset.
compartments	Compartment family used when inferring a built-in preset.
dynamic_beta	Logical scalar controlling whether the local transmission rate is time-varying for built-in presets.
dynamic_global	Logical scalar controlling whether the global rates are time-varying for built-in presets.
multi_outcome	Logical scalar indicating whether the observation model should include multiple outcomes.
model_type	Model family metadata attached to the returned specification. Either ‘"DecoderOnly"‘ or ‘"NeuralODE"‘.
time_varying_terms	Optional character vector overriding the default time-varying term metadata stored on the returned spec.
endogenous_terms	Optional character vector overriding the default endogenous term metadata stored on the returned spec.
tex_path	Optional path to a custom ‘.tex‘ file. When supplied, ‘ndm_model_spec()‘ delegates to ‘ndm_model_spec_from_tex()‘.
family_args	Optional named list used to expand parameterized model families such as chained latent-stage specifications. For structured TB presets, ‘n‘ controls supported chain lengths and ‘observation_target‘ chooses ‘"incidence"‘ (default flow into ‘i_1‘) or ‘"cumulative"‘ (‘i_1‘). TB presets use ‘dynamics_variant = "balanced_incidence"‘ by default to keep the original progression topology while adding demographic renewal, active-disease exit back to susceptibility, and incidence-flow observations. Set ‘dynamics_variant = "progression"‘ for legacy one-way depletion dynamics. Set ‘dynamics_variant = "seir"‘ for the SEIR-style form of ‘reports/TBModels/TBDiseaseTypes_v2.md‘: mass-action incidence ‘beta * S * I / N‘ replaces the constant force of infection ‘lambda‘, an explicit removed compartment ‘r_1‘ is added with active-disease removal ‘gamma‘ (I to R), and demographic renewal ‘mu‘ keeps incidence sustained (endemic) for fitting. Both ‘balanced_incidence‘ and ‘seir‘ require incidence observations.
path	Path to a ‘.tex‘ model specification file on disk.
spec	An ‘ndm_model_spec‘ object.
x	An ‘ndm_model_spec‘ object.
...	Unused; included for S3 method compatibility.

Details

For a maintainer-oriented walkthrough of preset selection, structured families, TeX round-tripping, and safe modification workflows, see `vignette("ode-structures", package = "ndm")`.

Value

‘ndm_model_spec_presets()‘ returns a data frame describing the built-in presets bundled with the package.

‘ndm_model_spec()‘ returns an object of class ‘ndm_model_spec‘. Built-in presets include metadata such as the packaged source path and the normalized TeX contents.

‘ndm_model_spec_from_tex()‘ returns an ‘ndm_model_spec‘ object whose ‘source_path‘ points at the supplied file. When the basename matches a built-in preset, the preset metadata is reused.

‘ndm_model_spec_to_tex()‘ returns the TeX text as a single character string when ‘path‘ is ‘NULL‘. When ‘path‘ is supplied, it writes the specification to disk and invisibly returns the normalized output path.

‘print.ndm_model_spec()‘ prints a brief summary of an ‘ndm_model_spec‘ object and invisibly returns ‘x‘.

Examples

```
ndm_model_spec_presets()

spec <- ndm_model_spec()
print(spec)

tb_spec <- ndm_model_spec(
  preset = "tb_b",
  model_type = "NeuralODE",
  family_args = list(n = 4L)
)
tb_spec$family_args

spec <- ndm_model_spec()
tex_path <- tempfile(fileext = ".tex")
ndm_model_spec_to_tex(spec, tex_path)
ndm_model_spec_from_tex(tex_path)

tb_spec <- ndm_model_spec(
  preset = "tb_k",
  model_type = "NeuralODE"
)
tb_tex_path <- tempfile(fileext = ".tex")
ndm_model_spec_to_tex(tb_spec, tb_tex_path)
ndm_model_spec_from_tex(tb_tex_path, model_type = "NeuralODE")

spec <- ndm_model_spec()
tex <- ndm_model_spec_to_tex(spec)
nchar(tex) > 0
```

---

ndm_new_runtime_env *Create and populate runtime environments*

Description

These helpers manage isolated environments used to load package-managed runtime code and seed it with R values.

Usage

```
ndm_new_runtime_env(parent = globalenv())

ndm_set_runtime_globals(env, values, overwrite = TRUE)
```

Arguments

parent	Parent environment for a new runtime environment. Defaults to ‘globalenv()‘ so package-managed runtime code can resolve attached-package functions consistently.
env	Runtime environment that should receive new bindings.
values	Named list of values to assign into ‘env‘.
overwrite	Logical scalar indicating whether existing bindings in ‘env‘ should be overwritten.

Value

‘ndm_new_runtime_env()‘ returns an environment of class ‘ndm_runtime_env‘. ‘ndm_set_runtime_globals()‘ invisibly returns ‘env‘.

Examples

```
env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
env$example_value
```

---

ndm_predict *Generate predictions or evaluate the training loss*

Description

These helpers call the runtime prediction and loss functions stored in a prepared runtime environment.

Usage

```
ndm_predict(x, batch = NULL, inference = TRUE, seed = 1L, update_state = TRUE)

ndm_loss(
  x,
  batch = NULL,
  y = NULL,
  y_mask = NULL,
  iteration = 1L,
  seed = 1L,
  update_state = TRUE
)
```

Arguments

x	Either an ‘ndm_model‘, an ‘ndm_trained_model‘, or a prepared runtime environment that already contains the required runtime objects.
batch	Optional batch object. Supply either a named TFRecord-style list or the four-element packaged list returned by ‘ndm_batch_to_model_inputs()‘. When ‘NULL‘, the runtime must already contain ‘batch_l_cal‘.

inference	Logical scalar indicating whether ‘ndm_predict()‘ should use the inference prediction path or the training prediction path.
seed	Integer seed used to derive the per-example JAX RNG keys.
update_state	Logical scalar indicating whether the runtime ‘state‘ object should be updated with the returned state.
y	Optional observed outcome tensor passed to ‘ndm_loss()‘. When omitted, ‘YTrue_out‘ is inferred from ‘batch‘.
y_mask	Optional outcome mask tensor passed to ‘ndm_loss()‘. When omitted, ‘YTrue_out_mask‘ is inferred from ‘batch‘.
iteration	Training iteration number forwarded to the runtime loss function.

Value

‘ndm_predict()‘ returns the prediction object produced by the runtime. ‘ndm_loss()‘ returns the loss object produced by the runtime.

Examples

```
## Not run:
preds <- ndm_predict(model, batch = batch_l_cal)
loss <- ndm_loss(model, batch = batch_l_cal)

## End(Not run)
```

---

ndm_prepare_runtime *Prepare a local runtime for execution*

Description

These wrappers load package-owned runtime components into an isolated environment and prepare the selected data workflow.

Usage

```
ndm_prepare_runtime(
  config = ndm_create_config(),
  runtime_env = ndm_new_runtime_env(),
  runtime_globals = list()
)

ndm_prepare_data(
  runtime_env,
  generator = c("sim", "real", "multidisease"),
  runtime_globals = list()
)
```

Arguments

config	An object of class ‘ndm_config‘, usually created by ‘ndm_create_config()‘.
runtime_env	Runtime environment that should receive the sourced objects.
runtime_globals	Named list of additional bindings to assign before the relevant runtime or data-preparation stage.
generator	Which data workflow to prepare in ‘runtime_env‘.

Details

Before building, training, predicting, or restoring artifacts from a runtime environment, initialize the backend for the target conda environment with ‘ndm_initialize_backend()‘. ‘ndm_prepare_runtime()‘ requires the ‘fastmatch‘ package. Canonical real and simulation preparation requires a completed ‘train_<BaseID>.tfrecord‘ / ‘inference_<BaseID>.tfrecord‘ pair and their ndmdatasets manifests. Set ‘SkipTfRecords = TRUE‘ only for explicit in-memory preparation; the simulation compatibility path then also requires ‘zoo‘. The compatibility fields ‘resave_tfrecords‘ and ‘ReSaveTfRecords‘ must remain ‘FALSE‘ in configs, runtime environments, and runtime globals. JAX precision is process-global; all models prepared in one R process must therefore use the same ‘float_type‘. Analysis2 runners reject selected grid rows that mix 32- and 64-bit precision before fitting starts.

Value

‘ndm_prepare_runtime()‘ invisibly returns ‘runtime_env‘ after loading the package-owned helper and backend code.

‘ndm_prepare_data()‘ invisibly returns ‘runtime_env‘ after preparing the requested data workflow.

Examples

```
env <- ndm_new_runtime_env()
cfg <- ndm_create_config()
## Not run:
ndm_prepare_runtime(cfg, env)

## End(Not run)

env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
```

---

ndm_print *Print a timestamped diagnostic message*

Description

‘ndm_print()‘ is a small logging helper used throughout the package-facing orchestration code.

Usage

```
ndm_print(text, quiet = FALSE)
```

Arguments

text	Text to print.
quiet	Logical scalar indicating whether output should be suppressed.

Value

The input ‘text‘, invisibly.

Examples

```
ndm_print("starting")
```

---

ndm_save_model *Save and restore model artifacts*

Description

These helpers persist the public ‘ndm_model‘ / ‘ndm_trained_model‘ objects to a versioned artifact directory, restore them into a fresh runtime environment, and resume training from the restored optimizer state.

Usage

```
ndm_save_model(x, path, bundle = NULL, overwrite = FALSE)

ndm_load_model(
  path,
  bundle = NULL,
  calibration_source = c("inference", "train")
)

ndm_resume_training(
  path,
  bundle = NULL,
  calibration_source = c("train", "inference"),
  train_globals = list()
)
```

Arguments

x	An object of class ‘ndm_model‘ or ‘ndm_trained_model‘.
path	Artifact directory path.
bundle	Optional ‘ndm_tfrecord_bundle‘ used to record TFRecord references for later resume workflows.
overwrite	Logical scalar indicating whether an existing artifact directory should be replaced.
calibration_source	Which dataset should supply the preview batch used to seed runtime globals such as ‘batch_l_cal‘ when ‘bundle‘ is attached.
train_globals	Optional named list of runtime globals applied immediately before ‘ndm_resume_training()‘ continues the training loop.

Details

These helpers assume ‘ndm_initialize_backend()‘ has already run for the active conda environment. ‘ndm_save_model()‘ only works for runtime-backed ‘ndm_model‘ / ‘ndm_trained_model‘ objects whose environments still contain the built model state, including ‘ModelList‘ and ‘state‘. Exact resume also requires saved optimizer state. When ‘bundle = NULL‘ in ‘ndm_resume_training()‘, the artifact metadata must already contain a recorded TFRecord bundle reference. ‘ndm_load_model()‘ and training-restore paths require the ‘rrapply‘ package.

Value

‘ndm_save_model()‘ returns the normalized artifact directory path, invisibly. ‘ndm_load_model()‘ returns an ‘ndm_model‘ or ‘ndm_trained_model‘ matching the saved artifact. ‘ndm_resume_training()‘ returns an ‘ndm_trained_model‘.

Examples

```
## Not run:
# After ndm_initialize_backend() and ndm_train() / ndm_fit():
artifact_dir <- ndm_save_model(trained_model, tempfile("ndm-artifact-"))
restored <- ndm_load_model(artifact_dir, bundle = tf_bundle)
resumed <- ndm_resume_training(artifact_dir, bundle = tf_bundle)

## End(Not run)
```

---

ndm_tfrecord_fields_real

Inspect TFRecord field contracts

Description

These helpers describe the field names and default TensorFlow dtypes expected by the packaged TFRecord readers.

Usage

```
ndm_tfrecord_fields_real()

ndm_tfrecord_fields_sim()

ndm_tfrecord_dtype_map(kind = c("real", "sim"))
```

Arguments

kind	TFRecord schema to inspect. Use ‘"real"‘ for observed-data records and ‘"sim"‘ for simulation records.

Value

‘ndm_tfrecord_fields_real()‘ and ‘ndm_tfrecord_fields_sim()‘ return character vectors of field names. ‘ndm_tfrecord_dtype_map()‘ returns a named character vector mapping each field to its default TensorFlow dtype.

Examples

```
ndm_tfrecord_fields_real()
ndm_tfrecord_dtype_map("sim")
```

`ndm_tf_batch_to_r` *Convert and bundle TFRecord batches*

Description

These helpers convert TensorFlow batches to native R or JAX objects, package them into the shape expected by the package-managed runtime, and attach dataset handles to a runtime environment.

Usage

```
ndm_tf_batch_to_r(batch)

ndm_tf_batch_to_jax(batch, backend = NULL)

ndm_batch_to_model_inputs(batch)

ndm_load_tfrecord_bundle(
  train_file,
  inference_file = NULL,
  batch_size,
  kind = c("real", "sim"),
  dtype_map = NULL,
  tensorflow = NULL,
  max_train_examples = NULL,
  max_inference_examples = NULL,
  shuffle_train = TRUE,
  shuffle_inference = FALSE
)

ndm_attach_tfrecord_bundle(
  env,
  bundle,
  backend = NULL,
  calibration_source = c("inference", "train")
)
```

Arguments

`batch`	A parsed TFRecord batch or a nested list containing TensorFlow tensors.
`backend`	Optional ‘ndm_backend‘ object. When omitted, ‘ndm_tf_batch_to_jax()‘ and ‘ndm_attach_tfrecord_bundle()‘ use the active backend from ‘ndm_backend_modules()‘ when available.
`train_file`	Path to the training TFRecord file.
`inference_file`	Optional path to an inference TFRecord file.
`batch_size`	Batch size used when constructing the training and inference datasets.

kind	TFRecord schema to use when reading 'train_file' and 'inference_file'.
dtype_map	Optional named dtype mapping passed through to the TFRecord readers.
tensorflow	Optional TensorFlow module. When omitted, ndm uses the active backend's TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time.
max_train_examples	Optional cap on the number of training examples to read.
max_inference_examples	Optional cap on the number of inference examples to read.
shuffle_train	Logical scalar indicating whether the training dataset should be shuffled.
shuffle_inference	Logical scalar indicating whether the inference dataset should be shuffled.
env	Runtime environment that should receive dataset handles and the preview calibration batch.
bundle	An object of class 'ndm_tfrecord_bundle' returned by 'ndm_load_tfrecord_bundle()'.
calibration_source	Which dataset should supply the preview batch used to seed runtime globals such as 'batch_1_cal'.

Value

'ndm_tf_batch_to_r()' recursively converts a TensorFlow batch to R arrays and lists. 'ndm_tf_batch_to_jax()' recursively converts tensors into JAX arrays. 'ndm_batch_to_model_inputs()' returns the packaged four-element list expected by the package-managed model stages. 'ndm_load_tfrecord_bundle()' returns an 'ndm_tfrecord_bundle' object. 'ndm_attach_tfrecord_bundle()' invisibly returns 'env'.

Examples

```
batch <- list(
  XPred = array(0, c(2, 3, 4)),
  XPred_mask = array(TRUE, c(2, 3, 4)),
  Context = array(0, c(2, 1, 4)),
  Context_mask = array(TRUE, c(2, 1, 4)),
  location_id_numeric = matrix(1L, nrow = 2),
  time_id_numeric = matrix(1L, nrow = 2)
)
packaged <- ndm_batch_to_model_inputs(batch)
length(packaged)
```

Index

ndm *(ndm-package)*, 2
ndm-package, 2
ndm_attach_tfrecord_bundle *(ndm_tf_batch_to_r)*, 31
ndm_backend_modules *(ndm_check_backend)*, 9
ndm_batch_to_model_inputs *(ndm_tf_batch_to_r)*, 31
ndm_bootstrap_multidisease_tfrecords, 3
ndm_bootstrap_real_tfrecords, 5
ndm_bootstrap_sim_tfrecords, 6
ndm_build_backend, *2*
ndm_build_backend *(ndm_check_backend)*, 9
ndm_build_model, *3*, 7
ndm_check_backend, 9
ndm_collect_tfrecord_batches *(ndm_create_tfrecord_parser)*, 19
ndm_create_config, *2*, 11
ndm_create_multidisease_run_config *(ndm_create_real_run_config)*, 14
ndm_create_real_run_config, *3*, 14
ndm_create_sim_run_config *(ndm_create_real_run_config)*, 14
ndm_create_tfrecord_parser, 19
ndm_extract_temporal_latents, 20
ndm_fit, *3*
ndm_fit *(ndm_build_model)*, 7
ndm_initialize_backend, *2*
ndm_initialize_backend *(ndm_check_backend)*, 9
ndm_latent_matrix *(ndm_extract_temporal_latents)*, 20
ndm_load_model, *3*
ndm_load_model *(ndm_save_model)*, 29
ndm_load_tfrecord_bundle, *3*
ndm_load_tfrecord_bundle *(ndm_tf_batch_to_r)*, 31
ndm_loss, *3*
ndm_loss *(ndm_predict)*, 26
ndm_model_spec, *2*
ndm_model_spec *(ndm_model_spec_presets)*, 23
ndm_model_spec_from_structure, 21
ndm_model_spec_from_tex, *2*
ndm_model_spec_from_tex *(ndm_model_spec_presets)*, 23
ndm_model_spec_presets, 23
ndm_model_spec_to_tex, *3*
ndm_model_spec_to_tex *(ndm_model_spec_presets)*, 23
ndm_new_runtime_env, 25
ndm_predict, *3*, 26
ndm_prepare_data, *3*
ndm_prepare_data *(ndm_prepare_runtime)*, 27
ndm_prepare_runtime, *3*, 27
ndm_print, 28
ndm_read_tfrecord_dataset, *3*
ndm_read_tfrecord_dataset *(ndm_create_tfrecord_parser)*, 19
ndm_resume_training, *3*
ndm_resume_training *(ndm_save_model)*, 29
ndm_run_multidisease, *3*
ndm_run_multidisease *(ndm_create_real_run_config)*, 14
ndm_run_real, *3*
ndm_run_real *(ndm_create_real_run_config)*, 14
ndm_run_sim, *3*
ndm_run_sim *(ndm_create_real_run_config)*, 14
ndm_save_model, *3*, 29
ndm_set_runtime_globals *(ndm_new_runtime_env)*, 25
ndm_tf_batch_to_jax *(ndm_tf_batch_to_r)*, 31
ndm_tf_batch_to_r, 31

ndm_tfrecord_dtype_map
(ndm_tfrecord_fields_real), 30
ndm_tfrecord_fields_real, 30
ndm_tfrecord_fields_sim
(ndm_tfrecord_fields_real), 30
ndm_train, *3*
ndm_train (ndm_build_model), 7

print.ndm_config (ndm_create_config), 11
print.ndm_model (ndm_build_model), 7
print.ndm_model_spec
(ndm_model_spec_presets), 23
print.ndm_trained_model
(ndm_build_model), 7